

LAB WORKBOOK

Assignment 7: Page Object Model (POM) Restructure

Tier 3 – Framework Design & Test Organization

1. Module Name and Experiment Title

Module: Unit Test Frameworks (Tier 3 – Framework Design & Test Organization)

Experiment: Page Object Model (POM) Restructure

2. Problem Statement

Take the working Selenium scripts from the earlier assignments and restructure them using the Page Object Model design pattern. Separate page-specific locators and UI interaction methods from the test cases so that test scripts contain the test flow and assertions, while page classes contain reusable Selenium operations.

3. Objective

To implement the Page Object Model in a Selenium Python framework and demonstrate separation of page locators, page actions, and test assertions for improved readability, reusability, and maintainability.

4. Tools, Software and Concepts Used

Category	Details
Programming Language	Python 3
Automation Tool	Selenium WebDriver
Test Framework	PyTest
Browser	Google Chrome
IDE/Editor	VS Code / PyCharm
Application Under Test	Automation Exercise (https://automationexercise.com/)
Design Pattern	Page Object Model (POM)
Concepts	Page classes, locators, reusable methods, WebDriverWait, assertions, separation of concerns

5. Source Code / Implementation

The following implementation separates Login and Products page operations into dedicated page classes.

5.1 Login Page – pages/login_page.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
class LoginPage:
```

```
    def __init__(self, driver):
```

```

        self.driver = driver
        self.wait = WebDriverWait(driver, 10)

# Locators
email_input = (By.CSS_SELECTOR, "input[data-qa='login-email']")
password_input = (By.CSS_SELECTOR, "input[data-qa='login-password']")
login_button = (By.CSS_SELECTOR, "button[data-qa='login-button']")
login_error = (By.XPATH, "//p[contains(text(),'incorrect')]")

def enter_email(self, email):
    self.wait.until(
        EC.visibility_of_element_located(self.email_input)
    ).send_keys(email)

def enter_password(self, password):
    self.wait.until(
        EC.visibility_of_element_located(self.password_input)
    ).send_keys(password)

def click_login(self):
    self.wait.until(
        EC.element_to_be_clickable(self.login_button)
    ).click()

def login(self, email, password):
    self.enter_email(email)
    self.enter_password(password)
    self.click_login()

def get_login_error(self):
    return self.wait.until(
        EC.visibility_of_element_located(self.login_error)
    ).text

```

5.2 Products Page – pages/products_page.py

```

from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

```

```

class ProductsPage:

    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(driver, 10)

    products_link = (By.XPATH, "//a[@href='/products']")
    search_box = (By.ID, "search_product")
    search_button = (By.ID, "submit_search")
    searched_products = (By.CSS_SELECTOR, ".productinfo")

    def open_products(self):
        self.wait.until(
            EC.element_to_be_clickable(self.products_link)
        ).click()

    def search_product(self, product_name):
        box = self.wait.until(
            EC.visibility_of_element_located(self.search_box)
        )

```

```

        box.clear()
        box.send_keys(product_name)
        self.wait.until(
            EC.element_to_be_clickable(self.search_button)
        ).click()

    def is_product_displayed(self):
        products = self.wait.until(
            EC.visibility_of_all_elements_located(self.searched_products)
        )
        return len(products) > 0

```

5.3 Test Case – tests/test_login.py

```

from pages.login_page import LoginPage

def test_valid_login(driver):
    login_page = LoginPage(driver)

    login_page.login(
        "YOUR_VALID_EMAIL",
        "YOUR_VALID_PASSWORD"
    )

    assert "Logged in as" in driver.page_source

```

6. Implementation Steps

1. Create a separate pages folder for page-specific classes.
2. Create LoginPage and ProductsPage classes.
3. Define all page locators inside the corresponding page class.
4. Create reusable methods such as login(), open_products(), and search_product().
5. Keep the actual test cases inside the tests folder.
6. Call page-object methods from the test instead of locating elements directly in the test.
7. Keep assertions in the test layer so that page classes remain focused on UI interaction.
8. Execute the test through PyTest and verify the result.

7. Result

The Selenium test was successfully restructured using the Page Object Model. Page-specific locators and UI methods were separated from the test logic, and the test executed through the page-object methods.

8. Observation

POM keeps locators and browser interactions inside page classes while test cases focus on business flow and assertions. This reduces duplication and makes future locator changes easier to maintain.

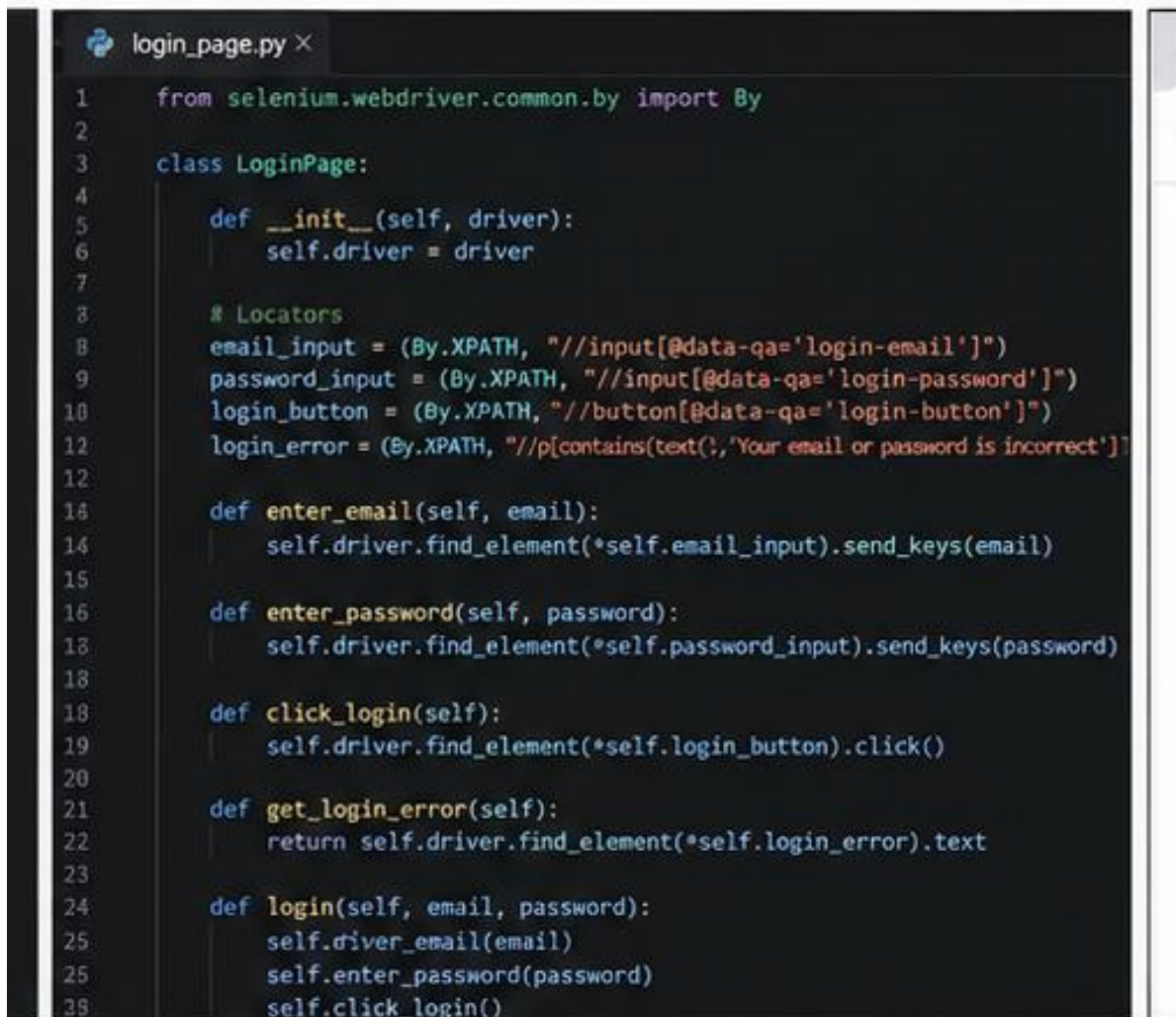
9. Conclusion

This experiment demonstrated how the Page Object Model can be used to organize a Selenium Python framework. Separating page objects from tests improves code readability, reuse, maintainability, and scalability.

10. Execution Evidence – Illustrative Screenshots

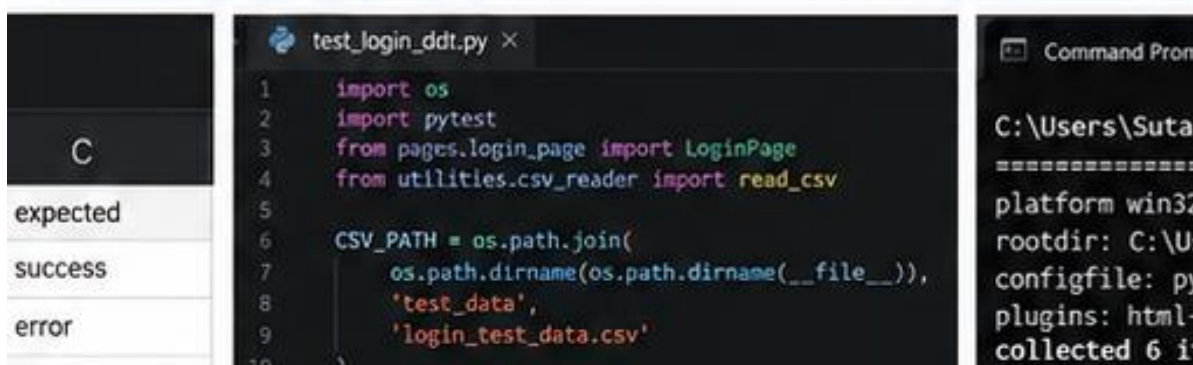
The following visuals are illustrative execution/documentation screenshots generated for the workbook. They should be replaced with actual screenshots from your own run if your institution requires real execution evidence.

Figure 1: LoginPage POM implementation.



```
login_page.py X
1  from selenium.webdriver.common.by import By
2
3  class LoginPage:
4
5      def __init__(self, driver):
6          self.driver = driver
7
8      # Locators
9      email_input = (By.XPATH, "//input[@data-qa='login-email']")
10     password_input = (By.XPATH, "//input[@data-qa='login-password']")
11     login_button = (By.XPATH, "//button[@data-qa='login-button']")
12     login_error = (By.XPATH, "//p[contains(text(),'Your email or password is incorrect')]")
13
14     def enter_email(self, email):
15         self.driver.find_element(*self.email_input).send_keys(email)
16
17     def enter_password(self, password):
18         self.driver.find_element(*self.password_input).send_keys(password)
19
20     def click_login(self):
21         self.driver.find_element(*self.login_button).click()
22
23     def get_login_error(self):
24         return self.driver.find_element(*self.login_error).text
25
26     def login(self, email, password):
27         self.enter_email(email)
28         self.enter_password(password)
29         self.click_login()
```

Fig 2: login_page.py (POM implementation)



```
test_login_ddt.py X
1  import os
2  import pytest
3  from pages.login_page import LoginPage
4  from utilities.csv_reader import read_csv
5
6  CSV_PATH = os.path.join(
7      os.path.dirname(os.path.dirname(__file__)),
8      'test_data',
9      'login_test_data.csv'
10 )

C
expected
success
error

Command Prompt
C:\Users\Sutara>
=====
platform win32
rootdir: C:\Users\Sutara
configfile: pyproject.toml
plugins: html-reporter, pytest-csv
collected 6 items
```

Figure 2: Automation Exercise login page.

Automation Exercise

automationexercise.com/login

 **AutomationExercise**

[Home](#) [Products](#) [Cart](#) [Signup / Login](#)

Login to your account

 validuser@example.com



Login

 Your email or password is incorrect!

[New User Signup!](#) [Forgot Password?](#)

Fig 3: Login Page on Automation Exercise

```
pt x + v - □ x
nu\Capstone_Project>pytest
===== test session starts =====
-- Python 3.13.1, pytest-9.1.1, pluggy-1.6.0
Sutanu\Capstone_Project
test.ini
4.2.0
ems
```